DeepShield AI - Technical Documentation

1. System Overview

DeepShield AI is a frontend application built to simulate forensic analysis of digital media (video, images, audio) for deepfake detection. Originally a monolithic HTML file, the application has been refactored into a modern, modular Vanilla JavaScript Single Page Application (SPA) powered by Vite.

2. Architecture & Tech Stack

- **Core Build Tool:** Vite (provides hot module replacement, optimized bundling).
- **Frontend Logic:** Modular Vanilla JavaScript (ES Modules).
- **Styling:** Tailwind CSS (utility-first styling), paired with custom CSS Variables for dynamic theming and glassmorphism.
- **Backend / Database / Auth:** Supabase (PostgreSQL, Authentication API).
- **Icons & Assets:** FontAwesome 6, Google Material Symbols, and custom SVGs.

3. Directory Structure

```
safeshield-vite/
├── index.html                    # Application shell and entry
point
├── package.json                  # Dependencies and build scripts
├── vite.config.js                # Vite bundler configuration
├── tailwind.config.js            # Tailwind theme and custom
colors mapping
├── public/
│   ├── logo.png                  # Website logo
│   └── ...                       # Static assets
└── src/
    ├── main.js                   # SPA router and initialization
logic
    ├── style.css                 # Global CSS variables and
utility classes
```

```
    ├── api/
    │   └── supabase.js          # Supabase client instance
    ├── store/
    │   └── state.js             # Global Application State
(AppState)
    ├── components/              # Javascript UI Logic modules
    │   ├── Auth.js              # Authentication, login, signup
    │   ├── FileUpload.js        # Drag-and-drop, file parsing,
analysis logic
    │   ├── Gauge.js             # SVG Gauge animation rendering
    │   ├── History.js           # Supabase history fetching and
rendering
    │   ├── Profile.js           # User profile state and UI
updates
    │   └── Waveform.js          # Canvas-based waveform
generation
    └── templates/               # UI HTML Strings
        ├── layoutTemplate.js    # Sidebar and Top Header
        ├── uploadTemplate.js    # Upload Dashboard (Home)
        ├── resultsTemplate.js   # Analysis Results & Metrics UI
        ├── historyTemplate.js   # History list view
        ├── profileTemplate.js   # User profile view
        └── authTemplate.js      # Authentication screen
```

4. Key Features & Implementation Details

4.1 SPA Routing mechanism

The application operates as a Single Page Application without a framework like React or Vue. It uses a lightweight custom router defined in `src/main.js`. * **Navigation:** The `navigate(pageId)` function hides all `.page` elements and adds the `.active` class to the requested view. * **Template Injection:** The UI is constructed dynamically by injecting template strings (`innerHTML`) from `src/templates/` into the `#app` container on initialization.

4.2 State Management

Global state is managed via the `AppState` object in `src/store/state.js`. This central store tracks user authentication status (`isAuthenticated`, `currentUser`), UI state (`currentPage`, `hasResults`), and current upload metrics.

4.3 Dark Mode Implementation

The application features a robust dark mode toggle that does not rely on hardcoded `dark:` Tailwind prefixes. Instead, `tailwind.config.js` maps theme colors (e.g., `surface`, `primary`) to standard CSS Variables. When the dark mode is toggled, a `.dark` class is applied to the root `<html>` element, which dynamically swaps the underlying hex values in `src/style.css` (based on Material Design 3 guidelines), instantly updating backgrounds, text, and SVG fills across the entire dashboard.

4.4 Simulation and Animations

To provide a premium feel, the analysis process simulates complex operations: * **Waveform (`Waveform.js`):** Dynamically draws an animated frequency visualizer on an HTML `<canvas>` using `requestAnimationFrame`. * **Verdict Gauge (`Gauge.js`):** Calculates dash offsets and rotational transforms to animate a speedometer-like SVG gauge based on the simulated deepfake probability score. * **Terminal Log (`FileUpload.js`):** Simulates a streaming command-line interface outputting forensic analysis steps.

4.5 Backend Integration (Supabase)

Supabase is used for user management and saving analysis history. * User registration and session tracking are handled via `supabase.auth`. * Analysis results (filename, type, score, confidence) are persisted to a PostgreSQL `history` table, which is then fetched and rendered in the History tab.

5. Development & Deployment

Running Locally: 1. Install dependencies: `npm install` 2. Start dev server: `npm run dev` 3. The server runs with Hot Module Replacement (HMR) at `http://localhost:3000`.

Building for Production: Run `npm run build`. Vite will bundle, minify, and optimize the JavaScript, CSS, and HTML into the `dist/` directory, ready to be deployed to any static host (Vercel, Netlify, GitHub Pages).